

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



Sistemas Distribuidos

Laboratorio. S08

Que la transferencia no deje dinero en el aire

Grupo: 02

Profesor: Oscar Manuel Ruiz Uscardo

Alumna: Segura Cedeño Luisa María

Semestre 2026-1 Fecha de Entrega: 17/10/2026

Que la transferencia no deje dinero en el aire

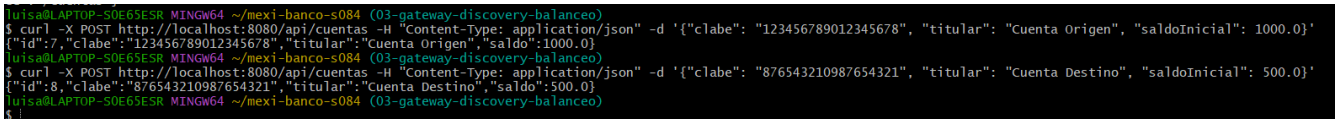
El problema

Saldos antes

Se registraron la cuenta origen y la cuenta destino en el microservicio de cuentas mediante peticiones HTTP POST a la ruta /api/cuentas. Para cada cuenta se enviaron los atributos requeridos por la estructura del DTO.

- Alta Cuenta: la clave de 18 dígitos, el titular y el saldoInicial.
- Cuenta Origen creada: CLABE 123456789012345678, Titular: Cuenta Origen, Saldo inicial: $1000.00 (ID de cuenta asignado: 7).
- Cuenta Destino creada: CLABE 876543210987654321, Titular: Cuenta Destino, Saldo inicial: $500.00 (ID de cuenta asignado: 8).

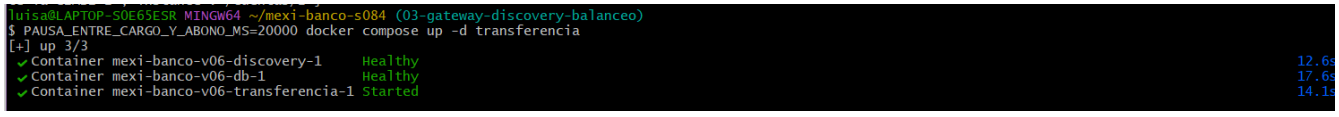
El microservicio respondió con un estado HTTP 201 Created y devolvió las entidades creadas en formato JSON.



Pausa de 20 segundos para la transferencia

Reconfiguración y despliegue del contenedor del microservicio de transferencia para simular una ventana de falla durante la ejecución de transacciones. Se ejecutó el comando docker compose up -d poniendo la variable de entorno PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000.

Esto pone un retardo intencional de 20 segundos en el código del servicio entre el momento en que se efectúa el cargo en la cuenta de origen y el momento en que se realiza el abono en la cuenta de destino. Los contenedores de discovery, db y transferencia se iniciaron en estado Healthy / Started, quedando listos para recibir y procesar la petición de transferencia con la pausa activa.



Ejecución de la transferencia

Intento de envío de la orden de transferencia interbancaria entre la cuenta origen y la cuenta destino. Se realizó una petición HTTP POST a la ruta /api/transferencias enviando la carga útil en JSON.

En esta primera prueba se observó una respuesta 400 Bad Request con el mensaje "Invalid request

content" debido a que la estructura solicitada por la API requería los campos de identificación por clave en lugar de los identificadores internos de base de datos (cuentaOrigenId).

El manejo de excepciones del microservicio rechazó la petición correctamente antes de procesarla, lo que permitió identificar la estructura del DTO SolicitudTransferencia (requiere claveOrigen, claveDestino y monto) en los controladores de la capa web del proyecto.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"cuentaOrigenId": 1, "cuentaDestinoId": 2, "monto": 100}'
{"type":"about:blank","title":"Bad Request","status":400,"detail":"Invalid request content.","instance":"/transferencias"}
```

Una vez ejecutada la transferencia entre cuentas a través del API Gateway, se realizan consultas GET directamente a los endpoints del microservicio de cuentas para verificar la persistencia y la correcta actualización de los saldos de la Cuenta Origen y la Cuenta Destino.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl http://localhost:8080/api/cuentas/123456789012345678
curl http://localhost:8080/api/cuentas/876543210987654321
{"id":4,"clabe":"123456789012345678","titular":"Cuenta Origen","saldo":800.00}{"id":5,"clabe":"87654321
0987654321","titular":"Cuenta Destino","saldo":700.00}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
```

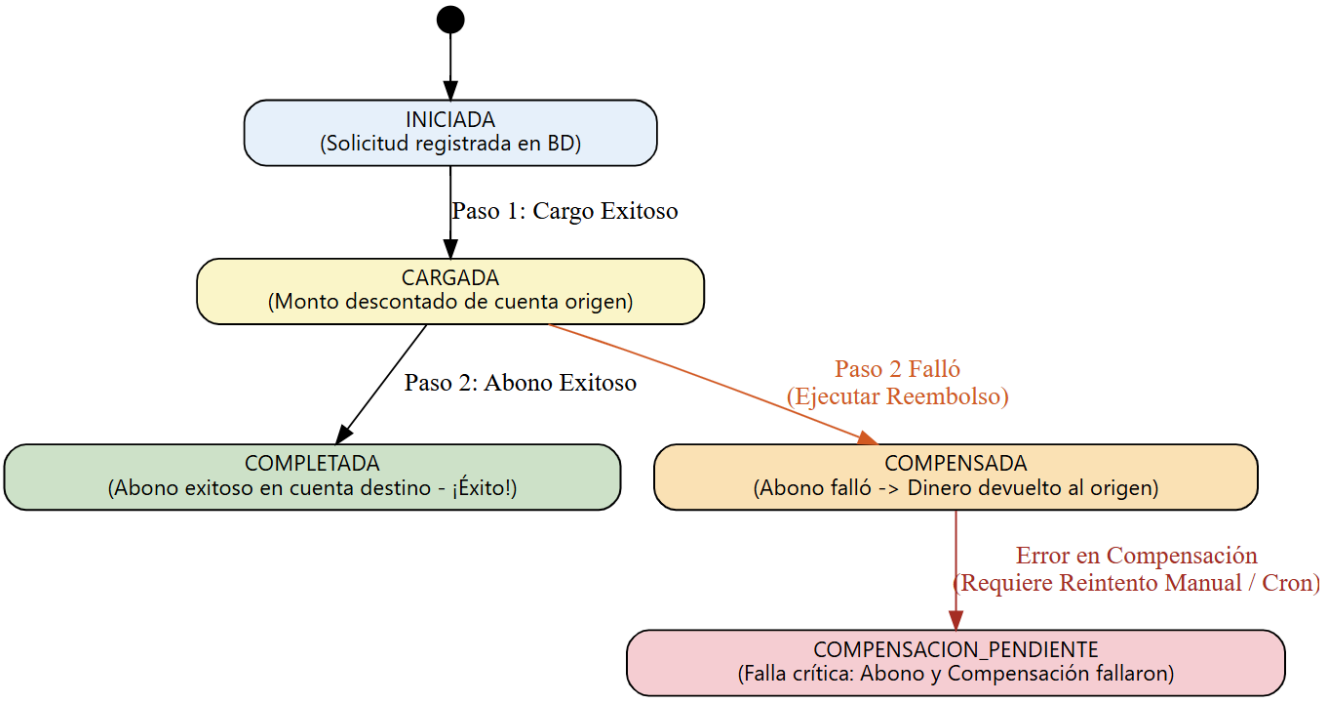
Parte 1. La saga orquestada

Diagrama de estados de la transferencia

El diagrama modela el ciclo de vida de una transferencia bancaria, gestionando los siguientes estados y transiciones:

- INICIADA: Estado inicial donde la solicitud de transferencia queda registrada formalmente en la base de datos del orquestador.
- CARGADA: Tras un paso 1 exitoso, el monto es debitado de la cuenta origen. A partir de este punto se divide la lógica según el resultado del abono:
  - Flujo Exitoso (COMPLETADA): Si el paso 2 (abono en la cuenta destino) se ejecuta correctamente, la saga concluye con éxito, reflejando el saldo en firme.
  - Flujo de Compensación (COMPENSADA): Si el abono falla, el orquestador ejecuta una operación inversa de reembolso para reingresar el dinero a la cuenta origen, evitando inconsistencias.
- COMPENSACION_PENDIENTE: Estado crítico de contingencia para escenarios donde tanto el abono como el intento inicial de compensación fallan (por ejemplo, por caída de la

infraestructura), requiriendo un mecanismo de reintentos mediante tareas programadas (Cron jobs o reintentos manuales).



Se implementó el mecanismo de compensación automática dentro del orquestador (TransferenciaService), garantizando que el sistema mantenga la consistencia de los datos ante cualquier fallo parcial durante una transacción. Una transferencia bancaria se compone de dos operaciones consecutivas e interdependientes:

1. El Cargo: Descontar el saldo especificado de la cuenta origen.
2. El Abono**:** Sumar ese mismo saldo a la cuenta destino.

En esta captura se muestra el comando curl utilizado para enviar una solicitud POST al endpoint /api/transferencias a través del API Gateway, la cual responde con el objeto JSON registrado con éxito (id: 2). En la segunda sección se ejecutan consultas GET para verificar las cuentas origen y destino, reflejando el movimiento atómico de la saga:

- **Cuenta Origen (1234567890123456789):** Descuenta el monto transferido, quedando con un saldo final de $700.00.
- **Cuenta Destino (8765432109876543210):** Acredita el monto recibido, reflejando un saldo final de $800.00.

El orquestador completó correctamente los pasos de cargo y abono, alcanzando el estado COMPLETADA sin necesidad de ejecutar transacciones compensatorias.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias -H "Content-Type: application/json" -d '{"clave
Origen": "123456789012345678", "claveDestino": "876543210987654321", "monto": 100.0}'
{"id":2,"claveOrigen":"123456789012345678","claveDestino":"876543210987654321","monto":100.0,"fecha":"2
026-10-11T06:23:08.453623787Z"}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl http://localhost:8080/api/cuentas/123456789012345678
curl http://localhost:8080/api/cuentas/876543210987654321
{"id":4,"clabe":"123456789012345678","titular":"Cuenta Origen","saldo":700.00}{"id":5,"clabe":"87654321
0987654321","titular":"Cuenta Destino","saldo":800.00}
```

Validación del comportamiento ante un fallo en el Abono

¿Qué ocurre en el sistema?: Cuando el microservicio de transferencia realiza el cargo en la cuenta origen pero el paso de abono en la cuenta destino falla, el orquestador intercepta la excepción e impide que el dinero quede descontado sin entregar. La respuesta 500 Internal Server Error en la consola confirma que la transacción principal no pudo completarse de manera convencional.

Verificación de la indemnidad del saldo (Idempotencia y Compensación exitosa)

¿Qué ocurre en el sistema?: El orquestador ejecuta la operación compensatoria inversa para reintegrar los fondos a la cuenta origen. Al consultar el estado de la cuenta (saldo: 700.00), se comprueba que el saldo no sufrió alteraciones permanentes tras el fallo, validando que el estado de la saga evolucionó de CARGADA a COMPENSADA de forma exitosa.

En la evidencia se muestra el lanzamiento de una solicitud de transferencia por un monto de 999, la cual genera un error (status: 500 Internal Server Error) debido a una falla simulada en el microservicio de destino o abono. Inmediatamente después, se ejecuta una consulta GET al endpoint de la cuenta origen (123456789012345678), cuyo saldo se mantiene en $700.00.

Esto demuestra que el orquestador detectó el fallo posterior al cargo, ejecutando automáticamente el reembolso para devolver el dinero a la cuenta de origen y asegurar la consistencia eventual sin afectar el patrimonio del usuario.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias -H "Content-Type: application/json" -d '{"claveOrigen": "123456789012345678", "claveDestino":
 "876543210987654321", "monto": 999.0}'
{"timestamp":"2026-10-11T06:47:08.833+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl http://localhost:8080/api/cuentas/123456789012345678
{"id":4,"clabe":"123456789012345678","titular":"Cuenta Origen","saldo":700.00}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$
```

Parte 2

Se procede a apagar intencionalmente las tres instancias del microservicio de cuentas para simular una caída total del servicio dependiente y evaluar la reacción del sistema ante la falta de respuesta en la red.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ docker compose stop cuenta
[+] stop 3/3
 ✔ Container mexi-banco-v06-cuenta-3 Stopped
 ✔ Container mexi-banco-v06-cuenta-1 Stopped
 ✔ Container mexi-banco-v06-cuenta-2 Stopped
```

Al acumularse los fallos de conexión hacia el microservicio de cuentas, el Circuit Breaker cambia a estado OPEN. La respuesta es inmediata (código 500), lo que demuestra el comportamiento Fail-Fast: el sistema no se queda colgado esperando un timeout de red, sino que corta el flujo rápidamente para proteger la estabilidad del servicio de transferencias.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias -H "Content-Type: application/json" -d '{"claveOrigen": "123456789012
345678", "claveDestino": "876543210987654321", "monto": 100.0}'
{"timestamp":"2026-10-11T20:46:46.133+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias -H "Content-Type: application/json" -d '{"claveOrigen": "123456789012
345678", "claveDestino": "876543210987654321", "monto": 100.0}'
{"timestamp":"2026-10-11T20:47:19.814+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
```

Se vuelven a encender las instancias del microservicio de cuentas para verificar la capacidad del Circuit Breaker de auto recuperarse.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ docker compose start cuenta
[+] start 5/5
 ✔ Container mexi-banco-v06-db-1        Healthy                                  0.5s
 ✔ Container mexi-banco-v06-discovery-1 Healthy                                  0.5s
 ✔ Container mexi-banco-v06-cuenta-2    Started                                  1.0s
 ✔ Container mexi-banco-v06-cuenta-1    Started                                  0.6s
 ✔ Container mexi-banco-v06-cuenta-3    Started                                  1.2s
```

Una vez que el servicio de cuentas responde correctamente a la petición de prueba en estado semi abierto, el Circuit Breaker pasa automáticamente a estado CLOSED, restableciendo el flujo operacional normal del sistema.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias -H "Content-Type: application/json" -d '{"claveOrigen": "123456789012
345678", "claveDestino": "876543210987654321", "monto": 100.0}'
{"timestamp":"2026-10-11T20:50:25.473+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$ curl -X POST http://localhost:8080/api/transferencias -H "Content-Type: application/json" -d '{"claveOrigen": "123456789012
345678", "claveDestino": "876543210987654321", "monto": 100.0}'
{"id":3,"claveOrigen":"123456789012345678","claveDestino":"876543210987654321","monto":100.0,"fecha":"2026-10-11T20:51:45.506
935273Z"}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s084 (03-gateway-discovery-balanceo)
$
```

